UESTC4036: Information Security

Report on Laboratory Session 2 & 3

Group Members:

Student Name:	GUID
Jiang haozhe	[redacted]
Chen Yichi	2939812

Statement of Contribution:

JiangHaozhe:

- Responsible for **Task 1 (Design of a Caesar Cipher)**, including the k-value calculation from the GUIDs and writing the caesar_encrypt function.
- Responsible for **Task 3 (Alphabet Frequency Deciphering)**, including coding the frequency counting (count_letter_frequency) and analysis plotting (frequency_analysis_plot) functions, and generating the frequency graphs.
- Authored the report sections, analysis, and screenshots for Task 1 and Task 3.

ChenYichi:

- Responsible for **Task 2 (Design of a Brute Force Decipher)**, including writing the caesar_decrypt function and the brute_force_caesar loop.
- Responsible for **Task 4 (Design of a Playfair Cipher)**, including coding the generate_playfair_matrix function.

- Authored the report sections, analysis, and screenshots for Task 2 and Task 4.

Joint Contributions:

- All members collaborated on debugging, integrating, and testing the final ultimate.m script.
- All members jointly discussed and wrote the **Introduction** and **Discussion** sections of the report.
- All members participated in the final formatting, proofreading, and submission of the report.

Introduction:

Write few sentences about the lab tasks. Briefly explain Caesar Cipher and Playfair Cipher.

This report covers the objectives and findings for Laboratory Sessions 2 and 3, which focus on the implementation and analysis of classical encryption techniques. The lab tasks required the design of encryption algorithms for the Caesar Cipher and the Playfair Cipher. Furthermore, we developed two distinct cryptanalysis methods for the Caesar Cipher: a brute-force deciphering tool and an attack based on alphabet frequency analysis. The report concludes with a discussion on the security limitations and comparative strengths of these two ciphers.

Caesar Cipher: The Caesar Cipher is a foundational substitution cipher where each letter in the plaintext is replaced by a letter a fixed number of positions down the alphabet. This fixed shift, denoted as the key *k*, is applied modulo 26. While simple to implement, its primary weakness is a very small key space, making it highly vulnerable to brute-force attacks.

Playfair Cipher: The Playfair Cipher is a polygraphic substitution cipher that encrypts pairs of letters (digraphs) instead of single letters. It uses a 5x5 matrix, generated from a secret keyword, to determine the substitution rules for each pair. By operating on digraphs, Playfair effectively masks the single-letter frequencies of the plaintext, offering greater security against simple statistical analysis than the Caesar Cipher.

Task 1: Design of a Caesar Cipher

Show your k-value calculation.

The k-value calculation was performed as follows:

1. First, the two GUIDs were added together:
    - guid1 = [redacted]
    - guid2 = [redacted]
    - sum_guid = [redacted] + [redacted] = 5679543
2. Next, the individual digits of the sum were added:
    - k_sum = 5 + 6 + 7 + 9 + 5 + 4 + 3 = 39

3. Finally, the modulo 26 of this sum was taken to determine the key *k*:
    - $k = \text{mod}(39, 26) = 13$

Therefore, the key *k* used for this experiment is **13**.

Paste your output cipher text here.

va guvf frpgvba naq gur arkg, jr rknzvar n fnzcyvat bs jung zvtug or pnyyrq pynffvpny rapelcgvba grpuavdhrf. n fghql bs gurfr grpuavdhrf ranoyrf hf gb vyyhfgengr gur onfvp nccebnpurf gb flzzrgevp rapelcgvba hfrq gbqnl naq gur glcrf bs pelcgnanylgvp nggnpxf gung zhfg or nagvpvcngrq. gur gjb onfvp ohvyqvat oybpxf bs nyy rapelcgvba grpuavdhrf ner fhofgvghgvba naq genafcbfvgvba. jr rknzvar gurfr va gur arkg gjb frpgvbaf. svanyyl, jr qvfphff n flfgrz gung pbzovarf obgu fhofgvghgvba naq genafcbfvgvba.n fhofgvghgvba grpuavdhr vf bar va juvpu gur yrggref bs cynvagrkg ner ercynprq ol bgure yrggref be ol ahzoref be flzobyf. vs gur cynvagrkg vf ivrjrq nf n frdhrapr bs ovgf, gura fhofgvghgvba vaibyirf ercynpvat cynvagrkg ovg cnggreaf jvgu pvcuregrkg ovg cnggreaf.

Answer the question: What happens to the encryption strength and ciphertext pattern when you increase k beyond 26?

Nothing changes to either the encryption strength or the ciphertext pattern, because the Caesar cipher relies on a modulo 26 operation.

Task 2: Design of a Brute Force Decipher for Caesar Cipher

Explain briefly how you solved this problem along with response to the questions asked in the lab description.

How we solved this problem:

We solved this problem by implementing the brute_force_caesar function, which leverages the fundamental weakness of the Caesar cipher: its extremely small key space.

- **Implementation:** The brute_force_caesar function contains a for loop that iterates through every possible key, from test_k = 1 to test_k = 26.
- **Decryption Attempts:** In each iteration, the loop calls the caesar_decrypt(ciphertext, test_k) function to decrypt the first 100 characters of the ciphertext using the current test_k.
- **Visual Inspection:** As required by the lab description, the program prints all 26 possible decrypted outputs. We then used "visual inspection" to find the single "legible output" that formed a meaningful English sentence.

Response to the questions asked in the lab description:

Question 1: What was the k-value you found?

- **Answer:** By visually inspecting the 26 outputs from the brute_force_caesar function, we identified that only the output for *k* = 13 produced legible English text (i.e., "in this section and the next..."). Therefore, the k-value found through brute force is 13.

Question 2: Does it match with your value you used for encryption?

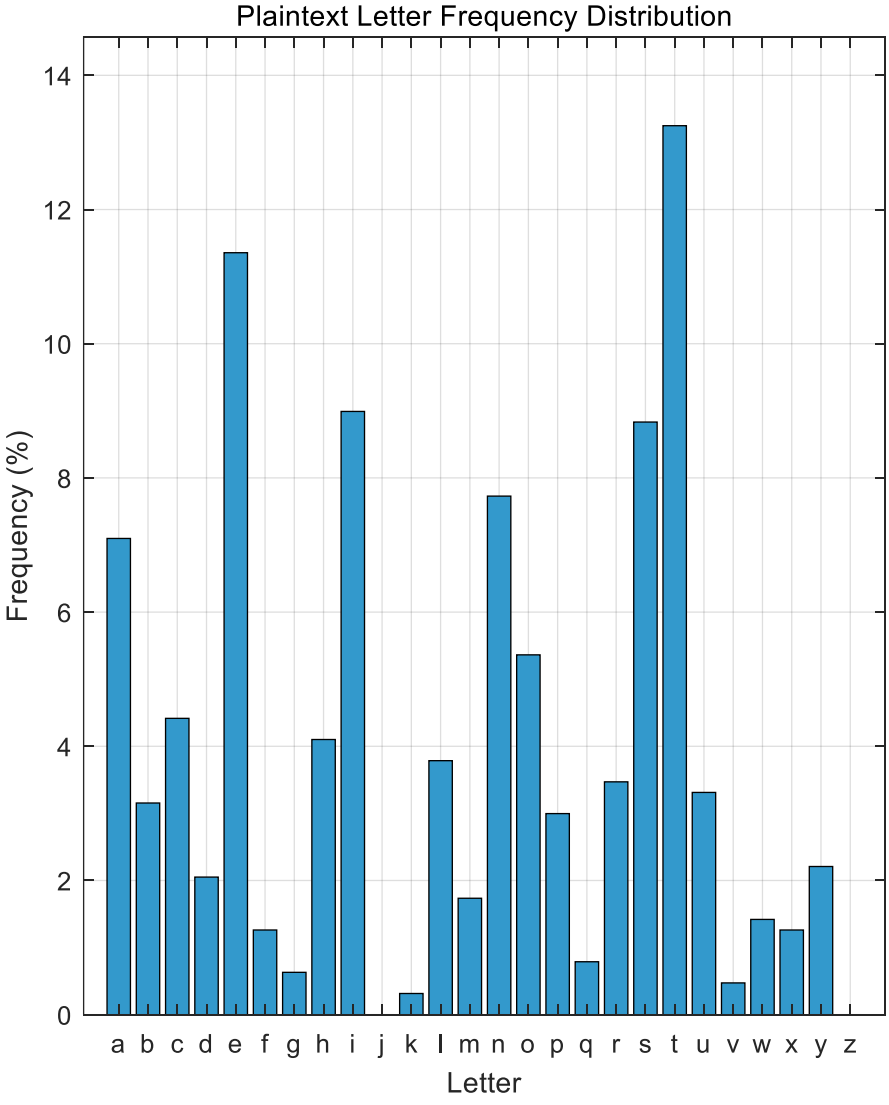
- **Answer:** Yes, it matches perfectly**.** The encryption key we calculated in Task 1 from our GUIDs was also 13. This confirms that the brute force cryptanalysis was 100% successful in recovering the correct key.

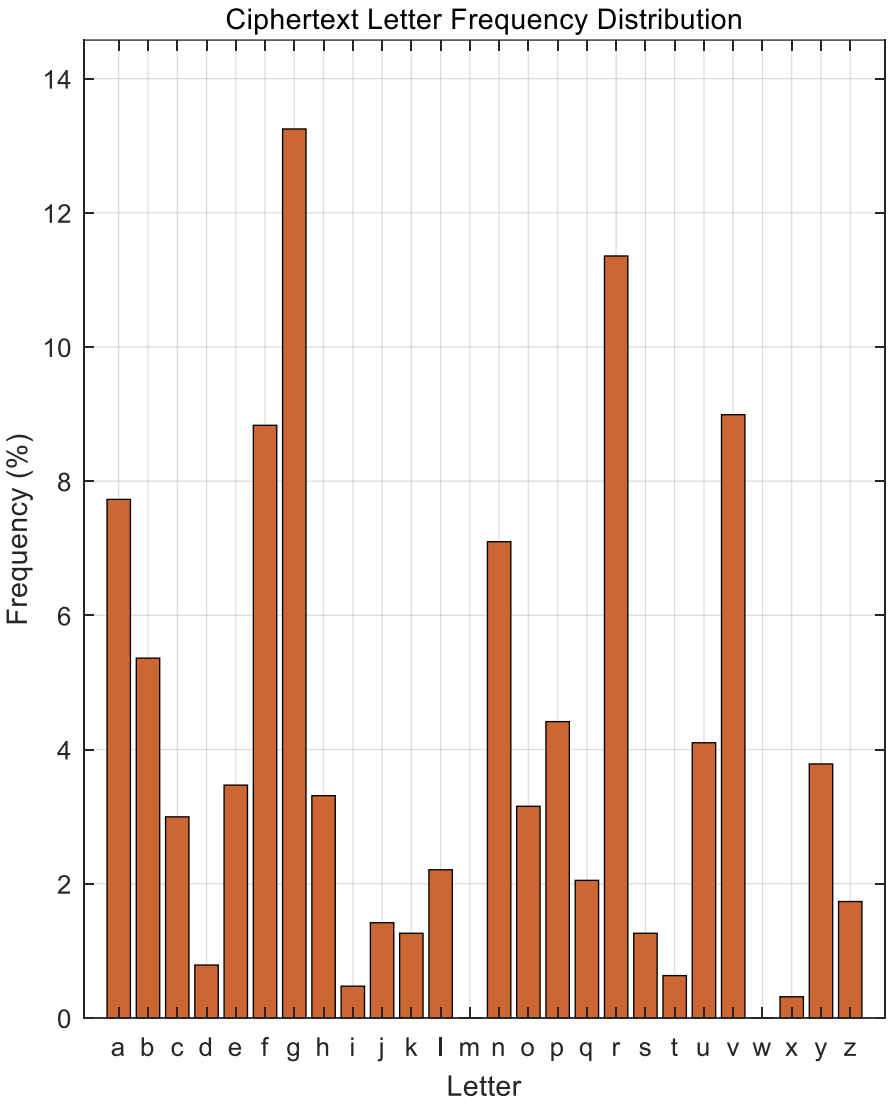
Task 3: Alphabet Frequency Deciphering:

Briefly explain how you approached this problem.

We approached this problem by writing a function (count_letter_frequency) that iterates through both the plaintext and the ciphertext, counting the occurrences of each letter from 'a' to 'z'. It then calculates the percentage frequency of each letter relative to the total number of letters. Another function (frequency_analysis_plot) was used to plot these two sets of frequencies as bar charts for visual comparison, as required by the lab instructions. The code also automatically finds the most frequent letter in the ciphertext and uses it to guess the key.

Can you plot the frequency of alphabets in your ciphertext and plaintext?





What was the most frequently occurred alphabet in ciphertext? How much was the frequency?

The most frequently occurred alphabet in the ciphertext was 'g'. Its frequency was 13.25%.

Did the most frequently occurred alphabet was sufficient to find the text?

Yes, it was sufficient. The frequency analysis method works by assuming the most frequent ciphertext letter corresponds to a common English letter, like 'e' or 't', Our code tested two hypotheses:

1. **Assuming 'g' mapped to 'e' (the most common letter):** This resulted in a wrong key guess of k=2. The decrypted text (ty estd dpnetzy...) was gibberish.
2. **Assuming 'g' mapped to 't' (the second most common letter):** This resulted in a correct key guess of k=13. The decrypted text (in this section and the next...) was the original, legible plaintext.

This succeeded because in our specific plaintext, 't' was the most common letter (13.25%), which became 'g' after k=13 encryption. Therefore, identifying the single most frequent letter was enough to find the correct key.

Briefly discuss your thoughts on impact of ciphertext length on accuracy of frequency-based key estimation.

The length of the ciphertext is critically important for the accuracy of this method.

Long Ciphertext (like ours): With a large sample of text, the letter frequencies will closely align with the known statistical averages of the language. This is why our attack was successful. The shape of our ciphertext frequency plot almost perfectly matched the plaintext plot, making the statistical link obvious.

Short Ciphertext: If the ciphertext were very short (e.g., just a few words), its letter distribution would be largely random and would not reflect the true frequencies of English. The most common letter in a short sample might be 'q' or 'z' purely by chance. In such a case, this frequency-based attack would fail, as it would be impossible to reliably map the ciphertext's most frequent letter to 'e' or 't'.

Task 4: Design of a Playfair Cipher

Please show your three Playfair matrix here.

Playfair matrix generated with keyword "exceeding":

EXCDI

NGABF

HKLMO

PQRST

UVWYZ

Playfair matrix generated with name "JiangHaozhe":

IANGH

OZEBC

DFKLM

PQRST

UVWXY

Playfair matrix generated with name "ChenYichi":

CHENY

IABDF

GKLMO

PQRST

UVWXZ

Does the choice of keyword influence the encryption strength?

No, the choice of keyword does not meaningfully influence the encryption strength. The keyword's only purpose is to generate the 5x5 matrix. The cipher's fundamental weakness is its vulnerability to digraph (two-letter) frequency analysis, and this attack works regardless of which keyword is used to arrange the matrix.

If an attacker knows your ciphertext but not the keyword, how could they attempt a cryptanalysis?

An attacker would use a method called digraph frequency analysis.

1. Count Digraphs: Instead of counting single letters (like in Task 3), the attacker counts the frequency of two-letter pairs (digraphs) in the ciphertext.
2. Compare Frequencies: The attacker compares this ciphertext digraph frequency list to a standard English digraph frequency list (e.g., 'TH', 'HE', 'IN', 'ER' are the most common).
3. Guess Mappings: They would assume the most frequent ciphertext digraph (e.g., 'XQ') corresponds to the most frequent English digraph (e.g., 'TH'). By making several such guesses, they can start to deduce the relative positions of letters in the 5x5 key matrix.

This attack is more complex than the single-letter frequency analysis used against the Caesar cipher, but it is a known and effective way to break Playfair, especially if the ciphertext is long enough to provide reliable statistics.

Discussion:

What is the main limitation of Caesar Cipher?

Based on the article [1], the main limitation of the Caesar Cipher is that it is "extremely easy to crack" and "offers no communication security".

The reasons for this limitation are:

- Fixed Substitution: In the Caesar cipher, "each character of a message is always replaced by the same fixed character that has been predetermined".
- Vulnerable to Attacks: The "simplicity of the algorithm makes it vulnerable to attacks". This is because the fixed substitution creates "repetitions," making it "easy to decrypt the encrypted information".

Considering the task 2 & 3, which method you think is better suited for deciphering. Explain your choice.

For the Caesar Cipher, Task 2 (Brute Force) is the superior method.

Explanation:

- Brute Force (Task 2) is guaranteed. It tests all 26 possible keys, which is instant for a computer and will always find the correct plaintext.
- Frequency Analysis (Task 3) is a statistical guess. It depends on the ciphertext being long enough to show a clear statistical pattern. It will fail if the message is too short.

Explain limitations of both the deciphering methods you have implemented.

A major limitation of Task 2 (Brute Force) is that it assumes only the correct key will produce readable plaintext, while all other keys will produce gibberish. However, modern cryptographic systems like "Honey Encryption" (HE) are designed specifically to counter this method [2]. When an attacker uses the wrong key to decrypt, the HE system "can generate valid looking plaintext" or "bogus messages". This renders our "visual inspection" method ineffective, as the attacker "doesn't know which message is true message or decoy message".

The primary limitation of Task 3 (Frequency Analysis) is that it only works on simple ciphers that fail to hide the statistical patterns of the plaintext. The attack relies on the fact that, in any language, every letter has its own unique frequency [3]. Our method was successful only because the Caesar cipher preserves this, where the same letter is repeatedly encrypted into the same ciphertext point [3].

Which cipher is more secure against statistical analysis, and why?

The Playfair cipher (Task 4) is more secure against statistical analysis.

- Why: The Caesar cipher (Task 3) is a simple monoalphabetic substitution. It preserves the exact frequency distribution of the plaintext (it just shifts it), making it extremely vulnerable to single-letter frequency analysis.
- The Playfair cipher is a polygraphic substitution that encrypts pairs of letters (digraphs). This breaks the single-letter statistics (e.g., the letter 'E' will be encrypted to different letters depending on what it is paired with), rendering single-letter frequency analysis completely ineffective.

Open discussion Questions: How might you modify playfair cipher to improve its security?

1.Use a Larger Matrix: Instead of a 5x5 matrix (which combines I/J), use a 6x6 matrix. This larger grid can hold all 26 letters plus the 10 digits (0-9), which both increases the complexity and allows for the encryption of numbers.

2. Use Multiple Matrices: Instead of one static matrix, generate two or more different matrices using different keywords. Then, alternate between them during encryption (e.g., use Matrix A for the first pair, Matrix B for the second, Matrix A for the third, and so on). This would make the cipher much more resistant to its primary weakness, digraph frequency analysis, as the same pair (like "TH") would encrypt to different results.

References:

Provide a list of references you have used to carry out your task and study of the relevant material. Make sure to cite them in your body of the report.

[1] S. N. Gowda, "Innovative enhancement of the Caesar cipher algorithm for cryptography," *2016 2nd International Conference on Advances in Computing, Communication, & Automation (ICACCA) (Fall)*, Bareilly, India, 2016, pp. 1-4, doi: 10.1109/ICACCAF.2016.7749010.

[2] K. S. M. Moe and T. Win, "Enhanced Honey Encryption Algorithm for Increasing Message Space against Brute Force Attack," *2018 15th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, Chiang Rai, Thailand, 2018, pp. 86-89, doi: 10.1109/ECTICon.2018.8620050.

[3] B. Padma, D. Chandravathi and L. Pratibha, "Defense Against Frequency Analysis In Elliptic Curve Cryptography Using K-Means Clustering," *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2021, pp. 64-69, doi: 10.1109/ICCCIS51004.2021.9397065.

Appendix:

Please copy your code here in text format. Please do not use image/screenshot of the code.

```
% Main Program - UESTC4036 Information Security Lab

clear; clc;


% ==================== Calculate Key k ====================

guid1 = [redacted];

guid2 = [redacted];

sum_guid = guid1 + guid2;

fprintf('Sum of GUIDs: %d\n', sum_guid);


% New calculation method: Sum of digits, then modulo 26

digits = num2str(sum_guid) - '0';

k_sum = sum(digits);

k_mod = mod(k_sum, 26);

fprintf('Sum of digits (k_sum) = %d\n', k_sum);

fprintf('k_mod = mod(%d, 26) = %d\n\n', k_sum, k_mod);


k = k_mod; % Use k_mod as the encryption key


% ==================== Main Program Execution ====================


% Plaintext

plaintext = ['In this section and the next, we examine a sampling of what might be called classical ', ...

        'encryption techniques. A study of these techniques enables us to illustrate the basic ', ...
```

```
    'approaches to symmetric encryption used today and the types of cryptanalytic attacks ', ...
    'that must be anticipated. The two basic building blocks of all encryption techniques ', ...
    'are substitution and transposition. We examine these in the next two sections. Finally, ', ...
    'we discuss a system that combines both substitution and transposition.A substitution ', ...
    'technique is one in which the letters of plaintext are replaced by other letters or by ', ...
    'numbers or symbols. If the plaintext is viewed as a sequence of bits, then substitution ', ...
    'involves replacing plaintext bit patterns with ciphertext bit patterns.'];

% Task 1: Encryption
fprintf('=== Task 1: Caesar Cipher Encryption ===\n');
ciphertext = caesar_encrypt(plaintext, k);
fprintf('Full Ciphertext Result:\n');
disp(ciphertext);
fprintf('\n');

% Task 2: Brute Force Decryption
fprintf('=== Task 2: Brute Force Caesar Cipher (k=1 to 26) ===\n');
brute_force_caesar(ciphertext(1:100));
fprintf('\n');

% Task 3: Frequency Analysis and Bar Chart
fprintf('=== Task 3: Frequency Analysis and Bar Chart ===\n');
frequency_analysis_plot(plaintext, ciphertext);
fprintf('\n');

% Task 4: Playfair Matrix Generation
fprintf('=== Task 4: Playfair Matrix Generation ===\n');
fprintf('Playfair matrix generated with keyword "exceeding":\n');
matrix1 = generate_playfair_matrix('exceeding');
disp(matrix1);
fprintf('\n');
```

```
fprintf('Playfair matrix generated with name "JiangHaozhe":\n');
matrix2 = generate_playfair_matrix('JiangHaozhe');
disp(matrix2);
fprintf('\n');

fprintf('Playfair matrix generated with name "ChenYichi":\n');
matrix2 = generate_playfair_matrix('ChenYichi');
disp(matrix2);
fprintf('\n');
% Verify Encryption and Decryption
fprintf('=== Encryption/Decryption Verification ===\n');
test_text = 'hello world';
encrypted = caesar_encrypt(test_text, k);
decrypted = caesar_decrypt(encrypted, k);
fprintf('Original: %s\n', test_text);
fprintf('Encrypted: %s\n', encrypted);
fprintf('Decrypted: %s\n', decrypted);
fprintf('Decryption correct: %s\n', string(strcmp(test_text, decrypted)));

% ==================== Function Definitions ====================

function ciphertext = caesar_encrypt(plaintext, k)
   % Caesar encryption function
   plaintext = lower(plaintext);
   ciphertext = '';

   for i = 1:length(plaintext)
      char_i = plaintext(i);

      if char_i >= 'a' && char_i <= 'z'
```

```
        p = char_i - 'a';
        c = mod(p + k, 26);
        cipher_char = char('a' + c);
        ciphertext = [ciphertext, cipher_char];
    else
        ciphertext = [ciphertext, char_i];
    end
  end
end

function decrypted_text = caesar_decrypt(ciphertext, k)
  % Caesar decryption function
  ciphertext = lower(ciphertext);
  decrypted_text = '';

  for i = 1:length(ciphertext)
    char_i = ciphertext(i);

    if char_i >= 'a' && char_i <= 'z'
        c = char_i - 'a';
        p = mod(c - k, 26);
        if p < 0
            p = p + 26;
        end
        decrypted_char = char('a' + p);
        decrypted_text = [decrypted_text, decrypted_char];
    else
        decrypted_text = [decrypted_text, char_i];
    end
  end
end
```

```
function brute_force_caesar(ciphertext)

    fprintf('Starting Caesar Cipher Brute Force (k=1 to 26)...\n\n');

    for test_k = 1:26
        decrypted_text = caesar_decrypt(ciphertext, test_k);
        fprintf('k = %2d: %s\n\n', test_k, decrypted_text);
    end

    fprintf('Brute force complete. Please manually check which k-value produces meaningful text.\n');
    fprintf('Hint: k=13 should produce the meaningful English text.\n');
end

function frequency_analysis_plot(plaintext, ciphertext)
    % Frequency analysis and bar chart function
    fprintf('Starting frequency analysis...\n');

    % Count plaintext letter frequency
    plain_freq = count_letter_frequency(plaintext);
    cipher_freq = count_letter_frequency(ciphertext);

    % Display frequency statistics
    fprintf('\nPlaintext Letter Frequency Statistics:\n');
    for i = 1:26
        fprintf('%c: %.2f%%  ', 'a' + i - 1, plain_freq(i));
        if mod(i, 8) == 0
            fprintf('\n');
        end
    end
```

```
fprintf('\n\nCiphertext Letter Frequency Statistics:\n');
for i = 1:26
   fprintf('%c: %.2f%%  ', 'a' + i - 1, cipher_freq(i));
   if mod(i, 8) == 0
      fprintf('\n');
   end
end
fprintf('\n');

% Plot bar chart - Corrected X-axis labels
figure('Position', [100, 100, 1200, 600]);

% Create labels for each letter
letters = cell(1, 26);
for i = 1:26
   letters{i} = char('a' + i - 1);
end

subplot(1,2,1);
bar(plain_freq, 'FaceColor', [0.2 0.6 0.8]);
title('Plaintext Letter Frequency Distribution');
xlabel('Letter');
ylabel('Frequency (%)');
set(gca, 'XTick', 1:26, 'XTickLabel', letters,'XTickLabelRotation', 0);
grid on;
ylim([0, max([plain_freq, cipher_freq]) * 1.1]);

subplot(1,2,2);
bar(cipher_freq, 'FaceColor', [0.8 0.4 0.2]);
title('Ciphertext Letter Frequency Distribution');
xlabel('Letter');
```

```
    ylabel('Frequency (%)');
    set(gca, 'XTick', 1:26, 'XTickLabel', letters,'XTickLabelRotation', 0);
    grid on;
    ylim([0, max([plain_freq, cipher_freq]) * 1.1]);

    % Frequency analysis cryptanalysis
    [max_freq, max_index] = max(cipher_freq);
    most_frequent_char = char('a' + max_index - 1);
    fprintf('\nMost frequent letter in ciphertext: %c (%.2f%%)\n', most_frequent_char, max_freq);

    % Assuming most common English letter is 'e', calculate k
    k_guess_e = mod((most_frequent_char - 'a') - ('e' - 'a'), 26);
    fprintf('Assuming %c corresponds to e, guessed k = %d\n', most_frequent_char, k_guess_e);

    % Assuming second most common English letter is 't', calculate k
    k_guess_t = mod((most_frequent_char - 'a') - ('t' - 'a'), 26);
    fprintf('Assuming %c corresponds to t, guessed k = %d\n', most_frequent_char, k_guess_t);

    % Decrypt using the guessed k values
    fprintf('\nDecrypting with k values from frequency analysis:\n');
    decrypted_e = caesar_decrypt(ciphertext(1:100), k_guess_e);
    fprintf('k=%d (from e): %s\n\n', k_guess_e, decrypted_e);

    decrypted_t = caesar_decrypt(ciphertext(1:100), k_guess_t);
    fprintf('k=%d (from t): %s\n\n', k_guess_t, decrypted_t);

    fprintf('Frequency analysis complete. Please compare the bar charts with general English
frequency distribution.\n');
end

function frequency = count_letter_frequency(text)
```

```
    % Function to count letter frequency
    letter_count = zeros(1, 26);
    total_letters = 0;

    text = lower(text);
    for i = 1:length(text)
        char_i = text(i);
        if char_i >= 'a' && char_i <= 'z'
            index = char_i - 'a' + 1;
            letter_count(index) = letter_count(index) + 1;
            total_letters = total_letters + 1;
        end
    end

    frequency = (letter_count / total_letters) * 100;
end

function playfair_matrix = generate_playfair_matrix(keyword)
    % Playfair matrix generation function
    keyword = upper(strrep(keyword, ' ', ''));
    keyword = strrep(keyword, 'J', 'I');
    alphabet = 'ABCDEFGHIKLMNOPQRSTUVWXYZ';

    matrix_str = '';

    % Add keyword letters (remove duplicates)
    for i = 1:length(keyword)
        current_char = keyword(i);
        if ~contains(matrix_str, current_char)
            matrix_str = [matrix_str, current_char];
        end
```

```
    end

    % Add remaining letters from the alphabet
    for i = 1:length(alphabet)
        current_char = alphabet(i);
        if ~contains(matrix_str, current_char)
            matrix_str = [matrix_str, current_char];
        end
    end

    % Reshape into 5x5 matrix
    playfair_matrix = reshape(matrix_str, 5, 5)';
end
```